

Submission: hw02

Zidane Karim <zidane.karim@cooper.edu>

11 Source Files

4 Plots

702 Lines

Generated: 2026-09-16 12:42:55

Source Code & Configuration

pyproject.toml

31 lines

```
1  # pyproject.toml.jinja
2
3  [project]
4  name = "hw02"
5  version = "0.1.0"
6  description = "Interweaving Spirals"
7  readme = "README.md"
8  authors = [
9      { name = "Zidane Karim", email = "zidane.karim@cooper.edu" }
10 ]
11 requires-python = ">=3.13"
12 dependencies = [
13     "structlog",
14     "rich",
15     "numpy",
16     "pydantic",
17     "pydantic-settings>=2.10.1",
18     "matplotlib>=3.10.5",
19     "tqdm>=4.67.1",
20     "jax[cpu]>=0.7.1",
21     "flax>=0.11.2",
22     "optax>=0.2.4",
23     "scikit-learn",
24 ]
25
26 [project.scripts]
27 hw02 = "hw02:main"
28 hw02-evaluate = "hw02.evaluate:main"
29 [build-system]
30 requires = ["uv_build>=0.8.3,<0.9.0"]
31 build-backend = "uv_build"
```

README.md

Markdown

3 lines

hw02

Interweaving Spirals: Decision Boundaries

src/hw02/config.toml

17 lines

```
1  debug = false
2  random_seed = 0xc61dd8fc
3
4  [data]
5  num_features = 1
6  num_samples = 50
7  sigma_noise = 0.1
8
9  [training]
10 batch_size = 16
11 num_iters = 3000
12 learning_rate = 0.01
13
14 [plotting]
15 output_dir = "artifacts"
16 figsize = [5, 3]
17 dpi = 200
```

src/hw02/__init__.py

93 lines

```

1  import jax
2  import numpy as np
3  import optax
4  import structlog
5  from flax import nnx
6
7  from .config import load_settings
8  from .data import Data
9  from .logging import configure_logging
10 from .model import LinearModel, NNXLinModel, SwiGLUMLP, MLP
11 from .plotting import plot_training_samples, plot_extrapolation
12 from .training import train
13
14
15 def main() -> None:
16     """CLI entry point."""
17     settings = load_settings()
18     configure_logging()
19     log = structlog.get_logger()
20     log.info("Settings loaded", settings=settings.model_dump())
21
22     # JAX PRNG
23     key = jax.random.key(settings.random_seed)
24     data_key, model_key = jax.random.split(key)
25     np_rng = np.random.default_rng(np.asarray(jax.random.key_data(data_key)))
26
27     # data_generating_model = LinearModel(
28     #     weights=np_rng.integers(low=0, high=5, size=(settings.data.num_features)),
29     #     bias=2,
30     # )
31     # log.debug("Data generating model", model=data_generating_model)
32
33     data = Data(
34         noise=settings.data.sigma_noise,
35     )
36     data.two_spiral_generator(rng=np_rng)
37
38     num_hidden_layers = 3
39     hidden_layer_width = 64
40     data_x, data_y = data.X, data.Y
41     model1 = MLP(
42         num_inputs=2, # x1, x2
43         num_outputs=2, #
44         num_hidden_layers=num_hidden_layers,
45         hidden_layer_width=hidden_layer_width,
46         hidden_activation=nnx.gelu,
47         output_activation=nnx.identity,
48         rngs=nnx.Rngs(params=model_key),
49     )
50
51     # log.debug("Initial model", model=model.model)
52
53     optimizer1 = nnx.Optimizer(
54         model1, optax.adam(settings.training.learning_rate), wrt=nnx.Param
55     )
56

```

```

57     train(model1, optimizer1, data, settings.training, np_rng)
58
59     # log.debug("Trained model", model=model.model)
60     plot_training_samples(model1, data, settings.plotting, "MLP")
61
62     model2 = SwiGLUMLP(
63         num_inputs=2, # x1, x2
64         num_outputs=2, #
65         num_hidden_layers=num_hidden_layers,
66         hidden_layer_width=hidden_layer_width,
67         output_activation=nnx.identity,
68         rngs=nnx.Rngs(params=model_key),
69     )
70
71     optimizer2 = nnx.Optimizer(
72         model2, optax.adam(settings.training.learning_rate), wrt=nnx.Param
73     )
74
75     train(model2, optimizer2, data, settings.training, np_rng)
76
77     plot_training_samples(model2, data, settings.plotting, "SwiGLU-MLP")
78
79     plot_extrapolation(model1, data, settings.plotting, "MLP")
80     plot_extrapolation(model2, data, settings.plotting, "SwiGLU-MLP")
81
82     print(
83         """
84         Explanation of cartesian boundary failure:
85         Models parameterized on cartesian coordinates "abandon" the spiral structure shape
86         soon after they exit the training envelope. This is because we directly fit onto x1,x2
87         coordinates. Even though pi exists, the model doesn't really have a concept of periodicity (e.g. 4pi
88         rotates to 2pi)
89         If we used purely sin or cos as points, the model could continue to model circular
90         features since the radius would repeat as theta keeps circling around, but since it is cartesian the model
91         tries to match its closest previous points
92         """
93     )
94     # if settings.data.num_features == 1:
95     #     plot_fit(model, data, settings.plotting)
96     # else:
97     #     log.info("Skipping plotting for multi-feature models.")

```

src/hw02/config.py

98 lines

```
1  from pathlib import Path
2  from importlib.resources import files
3  from typing import Tuple
4
5  from pydantic import BaseModel, Field
6  from pydantic_settings import (
7      BaseSettings,
8      PydanticBaseSettingsSource,
9      SettingsConfigDict,
10     TomlConfigSettingsSource,
11 )
12
13
14 def _find_config_toml() -> Path:
15     """Resolve config.toml location via importlib.resources or relative path fallback."""
16     if __package__:
17         try:
18             return Path(str(files(__package__).joinpath("config.toml")))
19         except Exception:
20             pass
21     return Path(__file__).parent / "config.toml"
22
23
24 class DataSettings(BaseModel):
25     """Settings for data generation."""
26
27     num_features: int = Field(
28         default=1, gt=0, description="Number of feature dimensions"
29     )
30     num_samples: int = Field(
31         default=50, gt=0, description="Number of synthetic data points"
32     )
33     sigma_noise: float = Field(
34         default=0.5, ge=0.0, description="Standard deviation of Gaussian noise"
35     )
36
37
38 class TrainingSettings(BaseModel):
39     """Settings for model training."""
40
41     batch_size: int = Field(default=16, gt=0, description="Mini-batch size for SGD")
42     num_iters: int = Field(
43         default=300, gt=0, description="Number of optimization iterations"
44     )
45     learning_rate: float = Field(
46         default=0.1, gt=0.0, description="Learning rate for Adam optimizer"
47     )
48
49
50 class PlottingSettings(BaseModel):
51     """Settings for plotting."""
52
53     figsize: Tuple[int, int] = (5, 3)
54     dpi: int = Field(default=200, gt=0, description="Dots per inch for saved figures")
55     output_dir: Path = Path("artifacts")
56
```

```
57
58 class AppSettings(BaseSettings):
59     """Main application settings."""
60
61     debug: bool = False
62     random_seed: int = 31415
63     data: DataSettings = DataSettings()
64     training: TrainingSettings = TrainingSettings()
65     plotting: PlottingSettings = PlottingSettings()
66
67     model_config = SettingsConfigDict(
68         toml_file=_find_config_toml(),
69         env_nested_delimiter="__",
70         extra="ignore",
71     )
72
73     @classmethod
74     def settings_customise_sources(
75         cls,
76         settings_cls: type[BaseSettings],
77         init_settings: PydanticBaseSettingsSource,
78         env_settings: PydanticBaseSettingsSource,
79         dotenv_settings: PydanticBaseSettingsSource,
80         file_secret_settings: PydanticBaseSettingsSource,
81     ) -> tuple[PydanticBaseSettingsSource, ...]:
82         """
83         Set the priority of settings sources.
84
85         We use a TOML file for configuration.
86         """
87         return (
88             init_settings,
89             TomlConfigSettingsSource(settings_cls),
90             env_settings,
91             dotenv_settings,
92             file_secret_settings,
93         )
94
95
96 def load_settings() -> AppSettings:
97     """Load application settings."""
98     return AppSettings()
```

src/hw02/data.py

50 lines

```

1  from dataclasses import InitVar, dataclass, field
2
3  import numpy as np
4
5  from .model import LinearModel
6
7
8  @dataclass
9  class Data:
10     """Handles generation of synthetic data for linear regression."""
11
12     noise: float
13     X: np.ndarray = field(init=False)
14     Y: np.ndarray = field(
15         init=False
16     ) # cuz we are not propagating this value beforehand
17
18     def two_spiral_generator(
19         self, turns: int = 2, points: int = 100, *, rng: np.random.Generator
20     ):
21         """Generates two spirals in order to avoid over-parameterizing this function"""
22         temp_radius = np.linspace(
23             1, 13, points
24         ) # roughly 13 from figure 2? this is a guess
25         radius = rng.normal(loc=temp_radius, scale=self.noise)
26         theta = np.linspace(0.0, turns * 2.0 * np.pi, points)
27         # Talked to Fisher Blumenfeld EE '28 & Josh Miao, and I think this version of plotting
28         # doesn't break assignment constraints
29         t0 = theta - np.pi / 2
30         x1_0 = radius * np.cos(t0)
31         x2_0 = radius * np.sin(t0)
32         set0 = np.column_stack(
33             [x1_0, x2_0]
34         ) # combines coordinates, sort of opposite of flatten
35
36         set1 = -set0
37
38         X = np.vstack([set0, set1])
39         Y = np.concatenate([np.zeros(points), np.ones(points)]).astype(
40             int
41         ) # had to search a bunch for concatenate function
42
43         idx = rng.permutation(len(Y))
44         self.X, self.Y = X[idx], Y[idx]
45
46     def get_batch(
47         self, rng: np.random.Generator, batch_size: int
48     ) -> tuple[np.ndarray, np.ndarray]:
49         """Select random subset of examples for training batch."""
50         choices = rng.choice(len(self.X), size=batch_size)
51         return self.X[choices], self.Y[choices].flatten()

```

src/hw02/model.py

156 lines

```

1  from dataclasses import dataclass
2
3  import jax
4  import jax.numpy as jnp
5  import numpy as np
6  from flax import nnx
7  from sklearn.base import BaseEstimator, ClassifierMixin
8
9
10 @dataclass
11 class LinearModel:
12     """Represents a simple linear model."""
13
14     weights: np.ndarray
15     bias: float
16
17
18 class SK_NNXClassifier(BaseEstimator, ClassifierMixin):
19     """Allows a Flax NNX model to be used with scikit-learn. DecisionBoundaryDisplay
20     builds a 2d grid of points and calls the model to get predictions for each point. This
21     wrapper is necessary because the model is not a scikit-learn model and does not have a predict method.
22     """
23
24     def __init__(self, model):
25         self.model = model
26         # 0, 1 because its either red or blue. this is used by DecisionBoundaryDisplay to
27         # determine the classes of the model.
28
29     def predict_proba(self, X):
30         """Returns the predicted class probabilities for the input data."""
31         X = jnp.asarray(X)
32         logits = self.model(X)
33         probs = jax.nn.softmax(logits, axis=-1)
34         return np.array(probs) # convert back to numpy array for sklearn
35
36     def predict(self, X):
37         proba = self.predict_proba(X)
38         return np.argmax(proba, axis=-1) # returns the index of the max probability
39
40     def fit(self, X, y=None):
41         self.classes_ = np.array([0, 1])
42         return self # had to do this because sklearn fails otherwise
43
44 class MLP(nnx.Module):
45     def __init__(
46         self,
47         num_inputs: int,
48         num_outputs: int,
49         num_hidden_layers: int,
50         hidden_layer_width: int,
51         hidden_activation=nnx.gelu,
52         output_activation=nnx.identity,
53         *,
54         rngs: nnx.Rngs,
55     ):

```

```

55     layers = []
56     self.hidden_activation = hidden_activation
57     self.output_activation = output_activation
58
59     layers.append(
60         NNLinearModel(rngs, num_inputs, hidden_layer_width)
61     ) # first layer hidden
62     for i in range(num_hidden_layers - 1):
63         layers.append(NNLinearModel(rngs, hidden_layer_width, hidden_layer_width))
64
65     self.hidden_layers = nnx.List(layers)
66     self.out_layer = NNLinearModel(rngs, hidden_layer_width, num_outputs)
67
68     def __call__(self, x: jax.Array) -> jax.Array:
69         for layer in self.hidden_layers:
70             x = self.hidden_activation(
71                 layer(x)
72             ) # at each layer, compute layer(x) = xW + b. then the hidden activation is
73             # applying the sigmoid or gelu. this then feeds as the next input (sort of recursive calls?)
74         return self.output_activation(
75             self.out_layer(x)
76         ) # take final-1 input, feed it to out_layer, and feed output_activation function to
77         # that
78
79     class SwiGLU(nnx.Module):
80         def __init__(self, num_inputs: int, num_outputs: int, *, rngs: nnx.Rngs):
81             self.W1 = NNLinearModel(rngs, num_inputs, num_outputs)
82             self.W2 = NNLinearModel(rngs, num_inputs, num_outputs)
83
84         def __call__(self, x: jax.Array) -> jax.Array:
85             gate = jax.nn.silu(self.W1(x))
86             return gate * self.W2(
87                 x
88             ) # element wise multiplication (gate symbol). see Shazeer section 2 equation 5
89
90     class SwiGLUWrapper(nnx.Module):
91         def __init__(self, dimension: int, *, rngs: nnx.Rngs):
92             self.norm = nnx.LayerNorm(
93                 dimension, rngs=rngs
94             ) # without normalization, SwiGLU loss is extremely high (50+)
95             self.swiglu = SwiGLU(dimension, dimension, rngs=rngs)
96
97         def __call__(self, x: jax.Array) -> jax.Array:
98             return (
99                 x + self.swiglu(self.norm(x))
100             ) # h = x + SwiGLU(X), see https://www.geeksforgeeks.org/deep-learning/residual-
101             # networks-resnet-deep-learning/ for where I saw resnet formula
102
103     class SwiGLUMLP(nnx.Module):
104         def __init__(
105             self,
106             num_inputs: int,
107             num_outputs: int,
108             num_hidden_layers: int,
109             hidden_layer_width: int,
110             output_activation=nnx.identity,
111             *,

```

```

112         rngs: nnx.Rngs,
113     ):
114         self.output_activation = output_activation
115         self.in_layer = NNLinearModel(
116             rngs=rngs, in_features=num_inputs, out_features=hidden_layer_width
117         )
118         layers = []
119         for i in range(num_hidden_layers):
120             layers.append(SwiGLUWrapper(hidden_layer_width, rngs=rngs))
121         self.layers = nnx.List(layers)
122         self.out_layer = NNLinearModel(
123             rngs=rngs, in_features=hidden_layer_width, out_features=num_outputs
124         )
125
126     def __call__(self, x: jax.Array) -> jax.Array:
127         x = self.in_layer(x)
128         for layer in self.layers:
129             x = layer(
130                 x
131             ) # x = X + SiLU(w1 x) * (w2 x), residual connection for each layer. same stuff as
MLP just formula change, and instead of hidden activation we just SiLU
132         return self.output_activation(self.out_layer(x))
133
134
135     class NNLinearModel(nnx.Module):
136         """A Flax NN module for a linear regression model, with multi-dimension support added to
solve MLP"""
137
138     def __init__(self, rngs: nnx.Rngs, in_features: int, out_features: int):
139         key = rngs.params()
140         self.in_features = in_features # bc of MLP
141         self.out_features = out_features
142         std = jnp.sqrt(1.0 / in_features) # prevents vanishing gradients
143         self.w = nnx.Param(jax.random.normal(key, (in_features, out_features)) * std)
144         self.b = nnx.Param(jnp.zeros((1, out_features))) # broadcasts for output
145
146     def __call__(self, x: jax.Array) -> jax.Array:
147         """Predicts the output for a given input."""
148         return x @ self.w.value + self.b.value # squeeze gone to maintain hidden layers
149
150     # @property
151     # def model(self) -> LinearModel:
152     #     """Returns the underlying simple linear model."""
153     #     return LinearModel(
154     #         weights=np.array(self.w.value).reshape([self.num_features]),
155     #         bias=float(np.array(self.b.value).squeeze()),
156     #     )

```

src/hw02/training.py

46 lines

```
1  import jax.numpy as jnp
2  import numpy as np
3  import structlog
4  from flax import nnx
5  from tqdm import trange
6
7  from .config import TrainingSettings
8  from .data import Data
9  from .model import NNLinearModel
10 import optax
11
12 log = structlog.get_logger()
13
14
15 @nnx.jit
16 def train_step(model, optimizer: nnx.Optimizer, x: jnp.ndarray, y: jnp.ndarray):
17     """Performs a single training step."""
18
19     def loss_fn(model):
20         logits = model(x)
21         return jnp.mean(optax.softmax_cross_entropy_with_integer_labels(logits, y))
22
23     loss, grads = nnx.value_and_grad(loss_fn)(model)
24     optimizer.update(model, grads) # In-place update of model parameters
25     return loss
26
27
28 def train(
29     model,
30     optimizer: nnx.Optimizer,
31     data: Data,
32     settings: TrainingSettings,
33     np_rng: np.random.Generator,
34 ) -> None:
35     """Train the model using SGD."""
36     log.info("Starting training", **settings.model_dump())
37     bar = trange(settings.num_iters)
38     for i in bar:
39         x_np, y_np = data.get_batch(np_rng, settings.batch_size)
40         x, y = jnp.asarray(x_np), jnp.asarray(y_np)
41
42         loss = train_step(model, optimizer, x, y)
43
44         bar.set_description(f"Loss @ {i} => {loss:.6f}")
45         bar.refresh()
46     log.info("Training finished")
```

src/hw02/evaluate.py

14 lines

```
1  """Evaluation entry point for hw02."""
2
3  import structlog
4
5  from .logging import configure_logging
6
7
8  def main() -> None:
9      """CLI entry point for homework evaluation."""
10     configure_logging()
11     log = structlog.get_logger()
12     log.info(
13         "Evaluation not yet implemented for hw02. Implement evaluation logic in src/hw02/
evaluate.py"
14     )
```

src/hw02/plotting.py

131 lines

```
1  import matplotlib
2  import matplotlib.pyplot as plt
3  import matplotlib.style
4  import jax.numpy as jnp
5  import numpy as np
6  import structlog
7  from sklearn.linear_model import (
8      LogisticRegression,
9  ) # this was on the docs so I included just in case
10 from sklearn.inspection import (
11     DecisionBoundaryDisplay,
12 ) # custom prediction functions for grids
13
14 from .config import PlottingSettings
15 from .data import Data
16 from .model import LinearModel, NNLinearModel, SK_NNXClassifier, MLP
17
18 ## Thanks Josh Miao EE '27 for confirming my plots
19
20
21 log = structlog.get_logger()
22
23 font = {
24     # "family": "Adobe Caslon Pro",
25     "size": 10,
26 }
27
28 matplotlib.style.use("classic")
29 matplotlib.rc("font", **font)
30 plt.rcParams["svg.fonttype"] = "none"
31
32
33 def plot_training_samples(
34     model,
35     data: Data,
36     settings: PlottingSettings,
37     instance: str,
38 ):
39
40     log.info("Plotting decision boundary")
41     fig, ax = plt.subplots(1, 1, figsize=settings.figsize, dpi=settings.dpi)
42
43     ax.set_title("Decision boundary")
44     ax.set_xlabel("x1")
45     h = ax.set_ylabel("x2", labelpad=10)
46     h.set_rotation(0)
47
48     clf = SK_NNXClassifier(model)
49
50     clf.fit(np.asarray(data.X), np.asarray(data.Y))
51
52     display = DecisionBoundaryDisplay.from_estimator(
53         clf,
54         np.asarray(data.X),
55         response_method="predict_proba",
56         class_of_interest=1, #  $P(t=1 | x)$ 
```

```

57         alpha=0.5,
58         grid_resolution=300,
59         cmap="RdBu", # red blue
60         ax=ax, # same plot?
61     )
62
63     ax.scatter(
64         np.asarray(data.X)[: , 0],
65         np.asarray(data.X)[: , 1],
66         c=np.asarray(data.Y).astype(int),
67         cmap="RdBu",
68         edgecolors="black",
69         s=15,
70     )
71
72     plt.tight_layout()
73     plt.title(instance)
74
75     settings.output_dir.mkdir(parents=True, exist_ok=True)
76     svg_path = settings.output_dir / f"decision_boundary_{instance}.svg"
77     pdf_path = settings.output_dir / f"decision_boundary_{instance}.pdf"
78     plt.savefig(svg_path)
79     plt.savefig(pdf_path)
80     plt.close(fig)
81     log.info("Saved plot", path=str(svg_path))
82
83
84     def plot_extrapolation(model, data, settings, instance):
85         log.info("Plotting decision boundary")
86         fig, ax = plt.subplots(1, 1, figsize=settings.figsize, dpi=settings.dpi)
87
88         ax.set_title("Decision boundary")
89         ax.set_xlabel("x1")
90         h = ax.set_ylabel("x2", labelpad=10)
91         h.set_rotation(0)
92
93         clf = SK_NNXClassifier(model)
94
95         clf.fit(np.asarray(data.X), np.asarray(data.Y))
96
97         clf_extended = np.max(np.abs(data.X)) * 2
98         points = np.array(
99             [[-clf_extended, -clf_extended], [clf_extended, clf_extended]]
100         ) # 2x2
101
102         display = DecisionBoundaryDisplay.from_estimator(
103             clf,
104             points,
105             response_method="predict_proba",
106             class_of_interest=1, # P(t=1 | x)
107             alpha=0.5,
108             grid_resolution=300,
109             cmap="RdBu", # red blue
110             ax=ax, # same plot?
111         )
112
113         ax.scatter(
114             np.asarray(data.X)[: , 0],
115             np.asarray(data.X)[: , 1],
116             c=np.asarray(data.Y).astype(int),

```

```
117         cmap="RdBu",
118         edgecolors="black",
119         s=15,
120     )
121
122     plt.tight_layout()
123     plt.title(f"{instance} Extrapolation")
124
125     settings.output_dir.mkdir(parents=True, exist_ok=True)
126     svg_path = settings.output_dir / f"decision_boundary_{instance}_extrapolation.svg"
127     pdf_path = settings.output_dir / f"decision_boundary_{instance}_extrapolation.pdf"
128     plt.savefig(svg_path)
129     plt.savefig(pdf_path)
130     plt.close(fig)
131     log.info("Saved plot", path=str(svg_path))
```

src/hw02/logging.py

63 lines

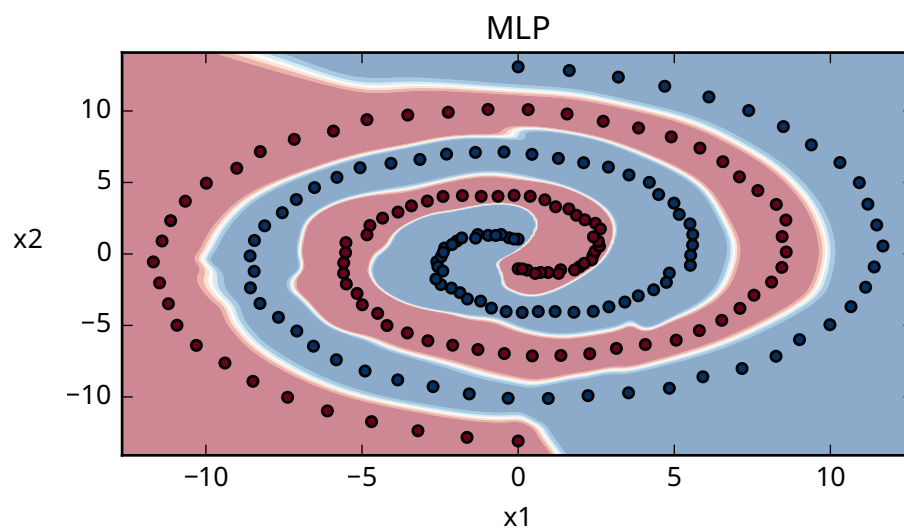
```
1  import logging
2  import os
3  import sys
4  from pathlib import Path
5
6  import jax
7  import numpy as np
8  import structlog
9
10
11  class FormattedFloat(float):
12      def __repr__(self) -> str:
13          return f"{self:.4g}"
14
15
16  def custom_serializer_processor(logger, method_name, event_dict):
17      for key, value in event_dict.items():
18          # Handle JAX arrays in addition to TF tensors
19          if hasattr(value, "numpy"): # Covers TF tensors
20              value = value.numpy()
21          if isinstance(value, jax.Array):
22              value = np.array(value)
23          if isinstance(value, (np.generic, np.ndarray)):
24              value = value.item() if value.size == 1 else value.tolist()
25          if isinstance(value, float):
26              value = FormattedFloat(value)
27          if isinstance(value, Path):
28              value = str(value)
29          event_dict[key] = value
30      return event_dict
31
32
33  def configure_logging():
34      """Configure logging for the application."""
35      log_level = os.environ.get("LOG_LEVEL", "INFO").upper()
36
37      logging.basicConfig(
38          format="%(message)s",
39          stream=sys.stdout,
40          level=log_level,
41          force=True,
42      )
43      logging.getLogger().setLevel(log_level)
44      structlog.configure(
45          processors=[
46              structlog.stdlib.filter_by_level,
47              structlog.stdlib.add_logger_name,
48              structlog.stdlib.add_log_level,
49              structlog.stdlib.PositionalArgumentsFormatter(),
50              structlog.processors.TimeStamper(fmt="iso"),
51              structlog.processors.StackInfoRenderer(),
52              structlog.processors.format_exc_info,
53              structlog.processors.UnicodeDecoder(),
54              custom_serializer_processor,
55              structlog.dev.ConsoleRenderer(
56                  colors=True, exception_formatter=structlog.dev.RichTracebackFormatter()
```

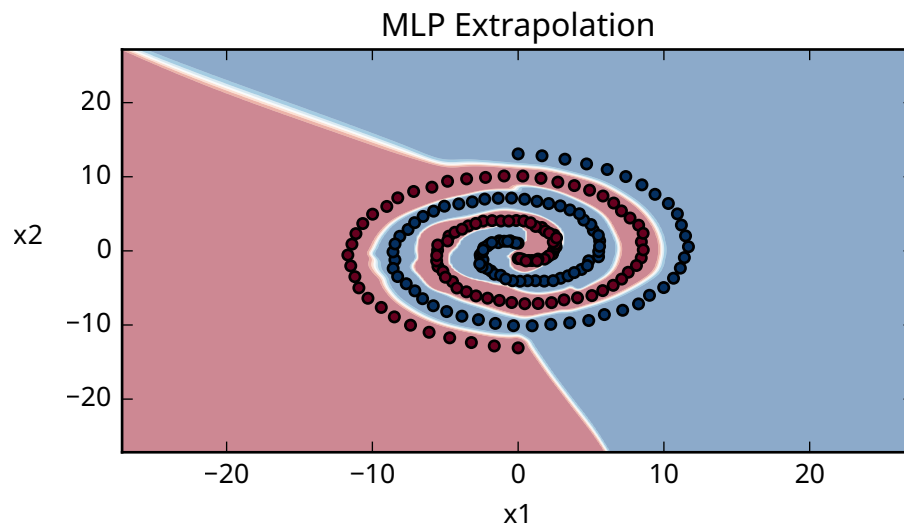
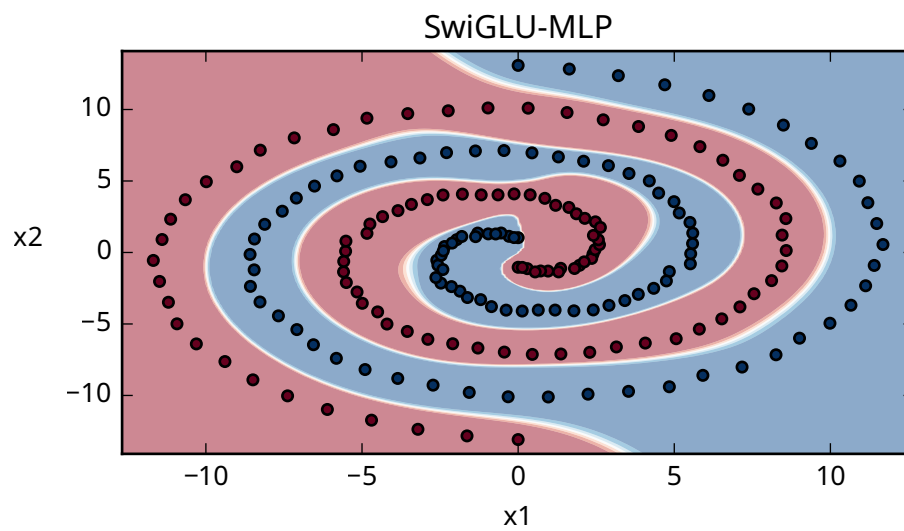
```
57         ),
58     ],
59     context_class=dict,
60     logger_factory=structlog.stdlib.LoggerFactory(),
61     wrapper_class=structlog.stdlib.BoundLogger,
62     cache_logger_on_first_use=True,
63 )
```

Artifacts & Generated Vector Plots

artifacts/decision_boundary_MLP.svg

[Vector SVG](#)



[artifacts/decision_boundary_MLP_extrapolation.svg](#)[Vector SVG](#)[artifacts/decision_boundary_SwiGLU-MLP.svg](#)[Vector SVG](#)

artifacts/decision_boundary_SwiGLU-MLP_extrapolation.svg

[Vector SVG](#)